Algorithmic Complexity Analysis

Theoretical Foundations, Space-Time Bounds, and Empirical Performance Across Laboratory Units

---

Course: IC-709D: Artificial Intelligence | **Academic Term:** Academic Year 2026–2027
Student Name: Jeet Verma | **Roll Number:** IC-2K23-46
Department: Department of Computer Science & Engineering
Institution: International Institute of Professional Studies (IIPS), Devi Ahilya Vishwavidyalaya (DAVV), Indore
Implementation Runtime: Python 3.8+ (Linux x86_64)

---

1. Executive Theoretical Summary

This document provides a rigorous, academically grounded complexity evaluation of the computational algorithms, search strategies, symbolic inference engines, decision tree models, neural optimization routines, and dimensionality reduction techniques implemented across the seven units of the **IC-709D: Artificial Intelligence** laboratory coursework. Each algorithmic paradigm is evaluated with respect to asymptotic time complexity, auxiliary memory requirements, branching factor (b), search depth (d), state-space cardinality ($|S|$), completeness guarantees, and optimality conditions.

A foundational distinction is maintained throughout: *theoretical complexity* establishes asymptotic upper bounds (O) under mathematical worst-case conditions, whereas *empirical observations* document the exact runtime dynamics and memory invariants observed in the verified laboratory Python implementations.

2. Master Complexity Comparison Across Laboratory Units

Lab Unit	Problem / Algorithm	Time Complexity	Space Complexity	Completeness	Optimality
Lab 1	Water Jug Problem (BFS)	O(|S| + |E|) = O(C_A · C_B)	O(|S|) = O(C_A · C_B)	Yes (Finite state space)	Yes (Min operations)
Lab 1	Vacuum Cleaner Agent (BFS)	O(|S|) = O(2 · 2^2) = O(8)	O(|S|) = O(8) states	Yes (Exhaustive)	Yes (Shortest actions)
Lab 1	Missionaries & Cannibals (BFS)	O(b^d) <= 32 valid states	O(|S|) = O(16) safe states	Yes (Graph search)	Yes (11 boat trips)
Lab 1/2	8-Puzzle Representation / BFS	O(b^d) with b <= 4, d <= 31	O(b^d) queue storage	Yes (Subspace |S|=9!/2)	Yes (Unit edge costs)
Lab 2	Graph Traversal (BFS)	O(V + E)	O(V) (Queue + Visited)	Yes (Reachable nodes)	Yes (Min hop count)
Lab 2	Graph Traversal (DFS)	O(V + E)	O(V) call stack	Yes (Finite graphs)	No (Arbitrary branch)
Lab 2	Grid Maze Solver (DFS)	O(R · C) grid cells	O(R · C) recursion stack	Yes (If path exists)	No (Explores deep)
Lab 2	Robot Navigation (BFS)	O(R · C) grid cells	O(R · C) queue memory	Yes (Finite 2D grid)	Yes (Shortest grid path)
Lab 2	Route Planning (BFS)	O(V + E) road network	O(V) frontier + visited	Yes (Connected road net)	Yes (Min segments)
Lab 3	Data Preproc. & Encoding	O(N · D) matrix scan	O(N · D') transformed matrix	Deterministic	Exact statistical fit
Lab 3	ANOVA F-Test & Mutual Info	O(D · N) & O(D · K · N log N)	O(N) working memory	Deterministic	Exact greedy ranking
Lab 4	Predicate Knowledge Base	O(1) assert, O(F) unification	O(F · A) ground assertions	Sound under CWA	Exact truth evaluation
Lab 4	Semantic Network Inheritance	O(V + E) DAG traversal	O(V + E) adjacency storage	Yes (Acyclic hierarchy)	Optimal path lookup
Lab 4	Frame Recursive Slot Engine	O(H · S) depth search	O(K · S) slot mappings	Yes (Root termination)	Nearest override priority
Lab 4	Forward Chaining Rule Engine	O(R · F^k) worst-case	O(F) working memory	Yes (Fixed point saturation)	Sound deduction trail
Lab 5	Decision Tree Classifier (CART)	Train: O(p · N log N · d)	O(2^d) tree nodes	Heuristic partition	Locally optimal (Gini)
Lab 6	PyTorch Autograd & Linear Reg.	O(E · N · W) per training run	O(W) params + O(N) DAG	Local minimum (SGD)	Optimal for convex MSE
Lab 6	MLP Non-Linear Classifier	O(E · N · sum(l_{i-1} l_i))	O(sum l_i) activations	Gradient convergence	Empirical local minimum
Lab 7	Grad-CAM Explainability	O(FLOPs_fwd + FLOPs_bwd)	O(C · H · W) feature maps	Deterministic attribution	Exact gradient weighting
Lab 7	FGSM Adversarial Attack	O(1) backward pass = O(FLOPs)	O(Input_size) perturbation	Yes (L_inf bounded)	Optimal linear step

Lab Unit	Problem / Algorithm	Time Complexity	Space Complexity	Completeness	Optimality
Lab 7	PCA & SVD Reduction	O(min(N^2 D, N D^2))	O(N · D) standardized array	Exact linear factorization	Optimal variance (Eckart-Young)

3. Search Algorithms: Mathematical Formalism & Trade-offs

Search algorithms formulate problem solving as finding a sequence of action operators that transform an initial state s_0 into a recognized goal state $s_g \in G$ within an implicitly or explicitly defined state graph $G = (V, E)$.

3.1 Breadth-First Search (BFS) Asymptotics

BFS expands states strictly in level-by-level (FIFO) order. In a uniform tree with branching factor b and shallowest goal depth d:

- **Time Complexity:** $\mathcal{T}_{\text{BFS}} = 1 + b + b^2 + \dots + b^d = O(b^d)$. Every node at depth d must be generated before the goal is verified.
- **Space Complexity:** $\mathcal{S}_{\text{BFS}} = O(b^d)$. The frontier queue holds all unexpanded leaf nodes, placing severe exponential pressure on physical RAM.
- **Completeness & Optimality:** BFS is unconditionally complete on finite branching factors ($b < \infty$). In unweighted graphs (where step cost $c = 1$), BFS is mathematically optimal, guaranteeing the shortest path length.
- **Laboratory Implementations:** Applied in Water Jug ($|S|=20$), Vacuum Cleaner ($|S|=8$), Missionaries & Cannibals ($|S|=16$ safe states), 8-Puzzle (solved in 2 moves), Graph Traversal (6 vertices), Robot Navigation (5×5 grid), and Route Planning (7 road junctions).

3.2 Depth-First Search (DFS) Asymptotics

DFS prioritizes vertical exploration along the current branch before backtracking via LIFO execution:

- **Time Complexity:** $\mathcal{T}_{\text{DFS}} = O(b^m)$, where m is the maximum depth of the state space. If $m \gg d$, DFS may explore vast redundant subtrees.
- **Space Complexity:** $\mathcal{S}_{\text{DFS}} = O(b \cdot m)$. Memory requirements scale linearly with search depth, representing a massive space advantage over BFS.
- **Completeness & Optimality:** DFS is complete on finite graphs with cycle detection (visited sets), but incomplete in infinite spaces. It is non-optimal, returning the first path discovered regardless of length.
- **Laboratory Implementations:** Used in Maze Solver (navigating grid obstacles to find valid escape routes) and Graph Traversal.

3.3 Theoretical Trade-offs: Uninformed vs. Informed Search

While the laboratory implementations focused strictly on uninformed search strategies (BFS and DFS) due to unweighted state graphs, classical AI theory highlights profound trade-offs when transitioning to informed (heuristic) search:

- **Uninformed (Blind) Search:** Operates without domain-specific distance metrics. It evaluates nodes solely based on graph connectivity ($g(n)$). Because it explores blindly in all spherical directions, frontier size grows exponentially ($O(b^d)$).
- **Informed (Heuristic) Search (A^* Search):** Incorporates evaluation function $f(n) = g(n) + h(n)$, where $h(n)$ estimates remaining cost to the goal. When $h(n)$ is admissible ($h(n) \le h^*(n)$) and consistent, A^* is optimal and maximally efficient, pruning the effective branching factor from b to $b^* \approx 1.1 - 1.5$.
- **Memory Trade-off:** Like BFS, standard A^* stores all open nodes in memory ($O(b^d)$), which can exhaust RAM on complex combinatorial problems. Memory-bounded variants (IDA*, SMA*) trade time for linear space ($O(b \cdot d)$).

4. Symbolic Knowledge Representation Complexity

Unit 4 implemented declarative reasoning systems governed by formal logic, associative graphs, and production rules:

- **Predicate Knowledge Base:** Ground fact assertions execute in $O(1)$ amortized time via hash-indexed sets. Unification queries with wildcard variables scan the fact partition $F_p \subseteq F$ in $O(|F_p|)$ linear time.
- **Semantic Networks:** Directed associative graphs represent relationships (`IS-A`, `HAS-A`, `PART-OF`). Taxonomic property inheritance performs depth-first upward traversal from instance to root, requiring $O(V + E)$ time and $O(V)$ stack space.

- **Frame Systems:** Minsky frame hierarchies resolve slot values through recursive prototype lookup. Time complexity is $O(H \cdot S)$, where H is inheritance tree height ($H \le 3$) and S is average slot count.
- **Forward-Chaining Rule Engine:** Matches rule premises against active working memory facts. In the worst case with R rules and premises of arity k, each match pass requires $O(R \cdot |F|^k)$. Because each derivation adds at least one new fact and memory is bounded, the engine guarantees polynomial termination at fixed-point saturation in at most $|F_{\text{max}}|$ iterations.

5. Machine Learning and Deep Learning Computational Bounds

Units 3, 5, 6, and 7 investigated continuous numerical algorithms, gradient-based optimization, and matrix factorizations:

- **Tabular Preprocessing & Leak-Free Pipelines (Lab 3):** Median imputation and standardization scan N rows and D columns in $O(N \cdot D)$ time. ANOVA F-tests require $O(D \cdot N)$ for between/within group sums of squares. Mutual information involves k-NN density estimation in $O(D \cdot K \cdot N \log N)$ using KD-trees.
- **CART Decision Tree Induction (Lab 5):** At each node, evaluating all continuous thresholds across p features requires sorting N samples, yielding $O(p \cdot N \log N)$ per split. For a balanced tree of depth d, total induction time is $O(p \cdot d \cdot N \log N)$. Inference time for an unseen test query is strictly $O(d)$, requiring only d scalar comparison operations.
- **PyTorch Dynamic Autograd & Optimization (Lab 6):** Forward propagation computes affine transformations $Z = WX + b$ in $O(M \cdot N \cdot K)$ FLOPs. Reverse-mode autograd traverses the tape-recorded DAG in reverse topological order, computing vector-Jacobian products in $O(\text{FLOPs}_{\text{forward}})$. Stochastic Gradient Descent updates parameters in $O(P)$ time and $O(1)$ auxiliary space, whereas Adam tracks first and second moment vectors, consuming $O(P)$ time and $2P$ additional state tensors.
- **Grad-CAM Explainability (Lab 7):** Visual attribution requires one forward pass ($O(\text{FLOPs}_{\text{CNN}})$), one class-specific backward pass to the target convolutional layer ($O(\text{FLOPs}_{\text{conv_back}})$), and Global Average Pooling across $K=32$ channels of size 8×8 ($O(K \cdot H \cdot W)$). Total computation is equivalent to a single training step, introducing near-zero inference latency.
- **FGSM Adversarial Attack (Lab 7):** Fast Gradient Sign Method executes in exactly $O(1)$ backward pass to the input tensor ($O(\text{FLOPs}_{\text{net}})$), extracting the sign vector and clamping in $O(C \cdot H \cdot W)$ operations. This linear single-step cost contrasts sharply with iterative attacks (e.g. PGD), which require K repeated gradient iterations.
- **PCA vs. SVD Equivalence (Lab 7):** Covariance-based PCA requires $O(N \cdot D^2)$ matrix multiplication followed by $O(D^3)$ eigen-decomposition. Thin SVD directly factorizes $X_{\text{std}} = U \Sigma V^T$ in $O(N \cdot D^2)$ time (assuming $N \ge D$). For $N=150$, $D=4$, computation executes in < 1 millisecond, achieving mathematical equivalence with residual error $\Delta < 4.0 \times 10^{-15}$.

6. Empirical Invariants and Verified Experimental Observations

The following empirical invariants were observed during programmatic execution of the laboratory implementations:

1. **8-Puzzle Path Length:** From initial state `[1 2 3 / 4 0 6 / 7 5 8]`, BFS explored 5 states and converged in exactly 2 moves (`Down`, `Right`) to reach `[1 2 3 / 4 5 6 / 7 8 0]`, demonstrating optimal shortest-path guarantee.
2. **Missionaries & Cannibals Invariant:** BFS explored 32 reachable state combinations, pruned 16 invalid states violating $M \ge C$, and returned the minimal safe sequence of 11 boat crossings.
3. **Decision Tree Overfitting Threshold:** In Lab 5, increasing tree depth from $d=2$ to unconstrained $d=10$ inflated training accuracy from 82.56% to 100.0%, while test accuracy collapsed from 79.07% to 69.77%, empirically confirming the theoretical variance penalty of excessive model depth.
4. **Neural Network Loss Convergence:** In Lab 6, linear regression MSE loss descended monotonically from 145.54 to 1.39 over 150 epochs under SGD ($\eta=0.03$), approaching the theoretical noise floor $\sigma^2 = 1.2^2 = 1.44$. The MLP reached 95.00% accuracy on non-linear synthetic moons data.
5. **Adversarial Robustness Collapse:** In Lab 7, introducing an FGSM perturbation budget of merely $\epsilon=0.05$ caused CIFAR-10 test accuracy to collapse from 38.0% to 4.0%, empirically validating the high-dimensional Linearity Hypothesis.
6. **PCA Variance Concentration:** In Lab 7, the first two principal components captured 72.96% and 22.85% of sample variance respectively, totaling 95.81% cumulative variance and enabling lossless 2D manifold visualization.

Conclusion: Algorithmic complexity governs both feasibility and scalability across symbolic, search, and statistical AI systems. Understanding these mathematical bounds is indispensable for designing performant, leak-free, interpretable, and robust intelligent systems.